A3 + The Slop Feedback Loop

Halley Young

March 4, 2026

1. Deliverable first

- `pip install a3-python` ships a static-first bug finding pipeline
- Stage 1: discover + prove/disprove + eliminate with auditable evidence
- Stage 2: send only unresolved residue to agentic triage
- Core design: deterministic elimination first, LLM judgment last

2. One ridiculous origin story (from the blog)

- Started as a challenge: can AI generate verification theory that survives implementation?
- Early output: elegant quantitative semantics, weak execution contact
- Project inversion: ideas survive only if executable and regression-stable
- Blog thesis: "fight AI slop with AI slop" $\rightarrow$ adversarial testing loop

3. Episode I: distance-to-safety intuition

- Quantitative view: safety as distance $d(\sigma, U)$, not only Boolean verdict
- Useful for prioritization and triage ordering
- Insufficient alone for production false-positive collapse
- Transition pressure: from ranking uncertainty to proving unreachability

4. Episode II: prove unsafe is unreachable

- Reachable set R, unsafe set U, target claim: $R \cap U = \varnothing$
- Synthesize separator certificate B with inductive side conditions
- If obligations hold, candidate bug is provably infeasible (high-confidence FP elimination)
- If obligations fail, route to refinement or alternate proof family

5. Barrier obligations (informal form)

Certificate constraints

$$\forall s \in S_0 : B(s) \geq \epsilon, \quad \forall s \to s' : B(s) \geq 0 \Rightarrow B(s') \geq 0, \quad \forall s \in U : B(s) \leq -\epsilon$$

- Initialization, consecution, unsafe separation must all hold
- Solver outcomes interpreted conservatively: UNSAT prune, SAT keep, UNKNOWN keep
- Obligation failure provides a refinement signal, not silent drop

6. One certificate mapped to one Python bug

1) Pick one concrete site
Example: DIV_ZERO at bytecode offset o for `num / den`.

↓

2) Build symbolic transition model
Choose state basis x from locals/stack/heap observers with path condition $\Phi(x)$.

↓

3) Define unsafe set
$U(x) \equiv (\mathtt{den}(x) = 0) \wedge \Phi(x)$.

↓

4) Choose template
$B(x) = \sum_{\alpha \in \mathcal{M}_k} c_\alpha x^\alpha$ with unknown coefficients.

↓

5) Solve + validate obligations
Enforce init/consecution/unsafe-separation; replay counterexamples to reject bad certificates.

↓

6) Map to finding status
Valid certificate $\Rightarrow$ unreachable FP with evidence; otherwise escalate to portfolio.

7. Episode III: Python reality constraints

- Bytecode-level control flow, not source-level wishful semantics
- Explicit normal and exceptional transitions (try/except/finally behavior)
- Unknown/library behavior modeled with conservative contracts
- Executable semantics is the filter that kills elegant but unsound theory

8. Short technical example from the blog

- Claim: "failing assertion escapes uncaught" is a reachability question
- Depends on assert-enable mode, enclosing handlers, caller handlers, finally edges
- Naive detector over-reports; theorem-only model under-specifies runtime
- Resolution pattern: precise-enough semantics + conservative proof obligations

9. The real engine: theorize $\rightarrow$ code $\rightarrow$ test $\rightarrow$ fix code $\rightarrow$ fix theory

1. AI theorizing proposes abstractions/proof templates quickly
2. Analyzer implementation exposes semantic and cost pathologies
3. Synthetic + regression + repo scans provide hard falsification pressure
4. Code fixes remove operational defects
5. Theory fixes tighten assumptions/obligations and restart the loop

10. What shipped: static-first, agentic-second

- Static stage performs bulk discovery, disproof/proof, dedup, confidence, SARIF
- Agentic stage inspects only unresolved survivors with repository context tools
- Design asymmetry: high-assurance methods consume high-volume workload
- LLM does not override upstream proof/disproof artifacts

11. Transition system used by the analyzer

- State $\sigma = (pc, \rho, h, \tau, \kappa, e, \Phi)$: control, env, heap, taint, stack, exception mode, path condition
- Step relation $\sigma \to \sigma'$ split by opcode class and edge type
- Control metadata stays concrete; value/flow uncertainty stays symbolic
- Unsafe predicate family $U_t(\sigma)$ indexed by bug type

12. What is concretely encoded

- Program counter per frame is concrete integer offset
- Locals/stack values are symbolic terms with type tags
- Heap observers (length/has-key/type predicates) become solver-visible terms
- Exception mode and handler context are first-class transition components

13. Kitchensink portfolio

- Principle: orchestrate strong methods; do not worship one paper
- Route by bug shape and cost: cheap-strong checks first, expensive methods later
- Families: barriers/SOS tiers, induction, CHC/PDR, CEGAR/interpolation, synthesis heuristics
- Method disagreement is signal; preserve orthogonal cross-checks

13b. Barrier obligations are the unifying interface

Every method answers one question about B

init : $\forall s \in S_0 : B(s) \geq \epsilon$ consec : $T \wedge B \geq 0 \Rightarrow B' \geq 0$ sep : $\forall s \in U : B(s) \leq -\epsilon$

- The barrier certificate defines **three provable obligations**
- Each paper in the portfolio is a specialised solver for one or more of these
- Composing them is coherent, not eclectic: every method either *validates* an obligation, *refutes* a candidate, or *narrows the template space*
- Failure of any single method is a signal to route to the next, not a project failure

13c. Separation obligation: foundations (papers 1–5)

Obligation: $\forall s \in U : B(s) \leq -\epsilon$ and $B(x) \geq 0$ on reachable region

- **Stengle 1974** (Positivstellensatz) — algebraic infeasibility certificate over semialgebraic sets;
  BSP: existence guarantee — if safe/unsafe are truly disjoint, a polynomial fence B must exist
- **Parrilo 2000** (SOS programming) — polynomial positivity reformulated as SDP via Gram matrix;
  BSP: the computational engine — turns "fence exists" into "we can solve for B's coefficients"
- **Lasserre 2001** (moment/SOS hierarchy) — iterative degree lifting to global optimum;
  BSP: if degree-d certificate fails, escalate to degree-$d+2$; hierarchy guarantees convergence
- **Prajna & Jadbabaie HSCC 2004** (barrier certificates) — first explicit init/consec/separation triple;
  BSP: this is the paradigm — they wrote the three-obligation API we use verbatim over symbolic state
- **Prajna, Jadbabaie & Pappas TAC 2007** (stochastic barrier) — consecution as expectation inequality;
  BSP: extends consecution to $E[B(s') \mid s] \geq 0$ for probabilistic transitions and hybrid paths

13d. Separation tractability: relaxation ladder (papers 6–8)

Problem: exact SOS/SDP is expensive at high degree or many variables

- **Ahmadi & Majumdar SIAM Rev. 2019** (DSOS/SDSOS) — LP/SOCP relaxations of SOS cone;
  BSP: B still satisfies all three obligations — the fence is sound, just certified faster via LP instead of SDP
- **Waki, Kim, Kojima & Muramatsu 2006** (correlative sparsity) — decompose SDP along variable correlation graph;
  BSP: Python variables rarely all interact; build B over small local clusters, not the full n-dimensional State space
- **Ames, Coogan, Egerstedt et al. CDC 2019** (CBF survey) — continuous consecution as $\dot{B} \geq -\alpha(B)$;
  BSP: discrete analogue is our transition consecution; CBF slope bound motivates contract overapproximation for unknown calls

Portfolio use: run DSOS tier first; escalate to full SOS/SDP only on failure.

13e. Consecution obligation: property-directed reachability (papers 9–12)

Obligation: $\forall s \to s' : B(s) \geq 0 \Rightarrow B(s') \geq 0$

- **Sheeran, Singh & Stålmårck FMCAD 2000** (k-induction) — consecution as bounded SAT base+step;
  BSP: checks "does B stay ≥ 0 for k steps then forever?" — unrolls induction instead of finding B explicitly
- **Bradley VMCAI 2011** (IC3) — inductive frame predicates block bad cubes forward;
  BSP: builds B implicitly as a clause conjunction; each blocked cube is one constraint maintaining the fence
- **Een, Mishchenko & Brayton FMCAD 2011** (PDR) — industrial IC3 with generalized counterexamples;
  BSP: same implicit B-as-clause-set at scale; inductive clause sequence serialises into our SARIF proof artifact
- **Gurfinkel, Kahsai, Komuravelli & Navas CAV 2015** (SeaHorn/CHC) — software safety as CHC;
  BSP: CHC solving is the consecution obligation written interprocedurally; tighter summaries = smaller CHC bodies

13f. Template refinement: CEGAR and interpolants (papers 13–16)

Problem: template B is wrong; counterexample says where

- **Clarke, Grumberg, Jha, Lu & Veith CAV 2000** (CEGAR) — spurious trace ⇒ refine abstraction;
  BSP: failed obligation = gap in the fence; CEGAR patches it by adding monomials to the template basis and re-solving
- **Ball, Majumdar, Millstein & Rajamani PLDI 2001** (SLAM) — CEGAR for C at Windows-driver scale;
  BSP: demonstrates that automated fence-gap patching works in real software without manual monomial selection
- **McMillan CAV 2003** (Craig interpolants) — tightest new predicate extracted from UNSAT proof;
  BSP: interpolant = the minimal new monomial closing the gap; derived directly from the failed obligation's UNSAT core
- **Henzinger, Jhala, Majumdar & Suresh POPL 2004** (lazy abstraction) — refine only the failing subtree;
  BSP: don't rebuild the entire B when one part fails — surgically patch only the template branch that broke

13g. Template synthesis & witness grounding (papers 17–20)

Problem: choose the right c_α and confirm with a concrete trace

- **Flanagan & Leino FME 2001** (Houdini) — optimistic invariant conjecture, prune by counterexample;
  BSP: propose many c_α configs; drop those that fail obligations; fixpoint = the widest valid B
- **Garg, Löding, Madhusudan & Neider CAV 2014** (ICE) — invariants from pos/neg/implication examples;
  BSP: concrete executions seed c_α values — the program's own behaviour shapes the fence coefficients
- **Alur, Bodik, Juniwal et al. FMCAD 2013** (SyGuS) — template grammar + SMT oracle loop;
  BSP: template family = the grammar; three obligation checks = the oracle; SyGuS formalises our coefficient search
- **Godefroid, Klarlund & Sen PLDI 2005** (DART/concolic) — concrete+symbolic path exploration;
  BSP: walks real executions to confirm $B \geq 0$ on live paths and $B \leq -\epsilon$ at real unsafe states

Key: papers 17–20 fill the "who proposes c_α" gap that pure SOS leaves open.

14. Solver and feasibility policy

- Candidate survives only if $\Phi \wedge U_t$ is feasible under current abstraction
- UNSAT: elimination artifact (why candidate is unreachable)
- SAT: retain candidate, optionally seed witness generation
- UNKNOWN/timeout: keep candidate conservatively; route to deeper stage

15. CEGAR-style refinement loop

- Start coarse to control compute cost
- Spurious counterexample $\Rightarrow$ refine predicates/contracts/context
- Recheck obligations under refined abstraction
- Stop on proof, witness, or budget boundary

16. Unknown calls: soundness vs precision

- Deterministic assumptions for unknown libraries are unsound
- Fully nondeterministic assumptions are too noisy
- Practical middle: conservative contracts + widening + concolic checks
- Precision improves as contracts mature; soundness envelope remains explicit

17. DSE/concolic role in the loop

- Symbolic stage filters and structures candidate space
- Concolic stage attempts concrete witnesses for unresolved SAT-like survivors
- Presence of witness raises actionability confidence
- Absence of witness is not safety proof (bounded exploration)

18. Agentic-second triage (residual only)

- Agent inspects source context, guards, callers, tests, imports
- Produces TP/FP rationale for uncertain residue
- This layer is intentionally downstream of proof-capable filtering
- Better static evidence produces better triage behavior

19. CI ratchet ("not a firehose")

- Baseline accepts historical findings once
- New unaccepted findings fail CI on protected branches
- Resolved findings are pruned automatically
- Policy meaning: monotonic "no net new risk" gate

20. Soundness envelope and known limits

- Python dynamism and reflection limit complete static precision
- Resource budgets force incompleteness on hardest programs
- Unknown external behavior depends on conservative contracts
- Claim is engineering-grade precision with explicit assumptions, not omniscience

21. Verification checklist

- What is proved: path infeasibility under encoded semantics + assumptions
- What is not proved: full language completeness or unlimited-resource reachability
- Why trust pruning: obligation traces + solver outcomes + reproducible artifacts
- Why trust escalation: conservative keep-on-UNKNOWN/timeout policy

22. Practical run modes

Single command

```
a3 scan .  --triage
```

Explicit staged mode

```
a3 scan .  --output-sarif results.sarif
a3 triage --sarif results.sarif --provider github --agentic
```

- Staged mode is preferred for CI observability and audit trails

23. Close + Q&A

- Architectural claim: static proof/disproof should absorb most decision volume
- Operational claim: triage only uncertain residue, never replace proof artifacts
- Process claim: CI ratchet enables sustainable rollout in legacy-heavy repos
- Open discussion: stronger contracts, nonlinear proof scalability, calibration drift